Assignment 3 in Algorithms Design

All additional code required by the assignment can be found in the original python file I attached. All that's required to run the code is to un-comment the desired lines (usually denoted as “#DP:”) in the main() function.

למי שבודק: מצאתי את זה כהזדמנות נחמדה לשפר את כתיבת האנגלית שלי, אבל התרגילים בעבודה גם דרשו המון חשיבה ואקספרמנטציה, שאלה החלקים שאני הכי אוהב בעבודות מהסוג הזה, ככה שאני לא אעלב ואפילו אציע שתתנו לבינה מלכותית לבדוק לי את העבודה כי מלבד הפתרונות עצמם יש פה המון מלל... הרגשתי צורך לתעד באופן מלא כל שלב בפתרון הבעיות, מתהליך הגישה עצמה לתרגילים ועד יישום הפתרון והניתוח שלו, במטרה להראות שבאמת ניסיתי ופתרתי את התרגילים האלה בעצמי. קישורים לשיחות ה-gemini נמצאים בסוף העבודה והם מעודכנים למצב האחרון שבהם השארתי את השיחות. גם חייב תודה אישית למי שבנה את העבודה, אלה היו תרגילים ממש טובים שאת חלקם הכרתי אבל חלקם הביאו לי המון הנאה לצלול לעומק וליהנות מתהליך החשיבה עצמו, שאני מאמין שהוא התהליך שמפתח את החשיבה ופתרון הבעיות בצורה הטובה ביותר.

Problem 1:

So my first instinct for this problem was to skip the thinking process entirely and simply jump into recreating a cute recursive exercise I've done before and know fairly well how to implement it in python:

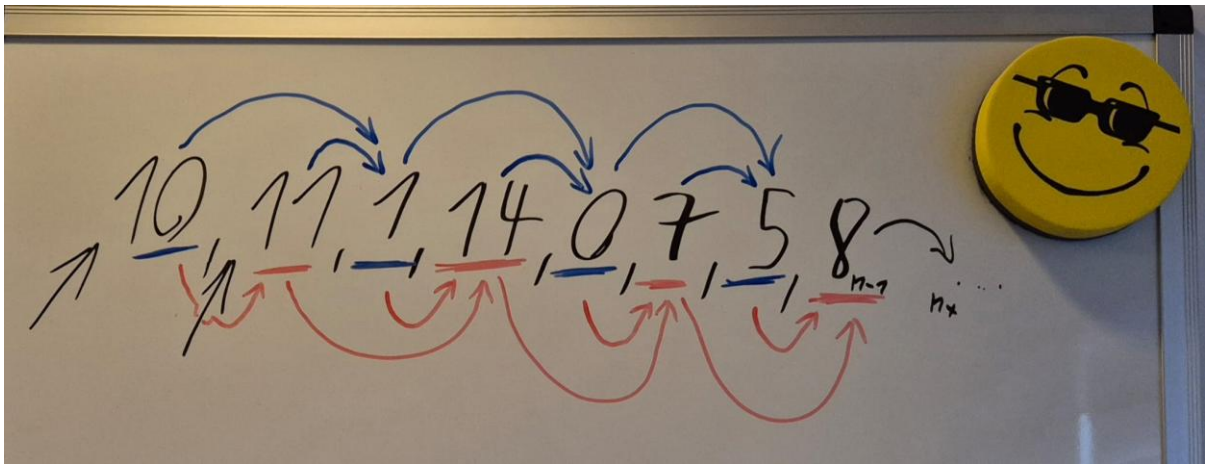
```
def targil_1(costs_list):
    function_calls = [0]
    cost_starting_at_0 = targil_1_rec(costs_list, 0, 0, function_calls)
    cost_starting_at_1 = targil_1_rec(costs_list, 0, 1, function_calls)
    return min(cost_starting_at_0, cost_starting_at_1), function_calls[0]


def targil_1_rec(costs_list, total_cost, current_location, function_calls):
    if len(costs_list)<=current_location: return total_cost

    total_cost += costs_list[current_location]
    function_calls[0] += 1
    #print(total_cost) discarded this part...
    return min(targil_1_rec(costs_list, total_cost, current_location + 1, function_calls),
               targil_1_rec(costs_list, total_cost, current_location + 2, function_calls))
```

and all was fun and well until I went to checking the simple benchmarking I've built which isn't actual benchmarking but simply just a counter that checks the overall recursive calls done within the program, and I saw "*46366", which totally caught me off guard, even though it makes perfect sense since a recursive function that splits into 2 other recursive instances has an exponential growth rate of 2^n and scales incredibly quickly.

(*P.S. if I moved the function_calls counter to the start of the recursive function, that would only blow up to "92734", as now we would also be counting the calls which pass the costs_list's bounds, and those calls are considered function calls too... not good for runtime, that's for sure...)

so that got me thinking how can I optimize the overall count of calls, which in this problem's case represents "the overall repeating actions of comparisons" in an algorithmics-course formal sense of the problem.

I drew the same example list as in the code and started thinking what operations am I repeating unnecessarily, which otherwise could be skipped, and might drastically improve the big-O-runtime of my algorithm:



After a fairly short drawing of the first 8 elements I've remembered a half-formed idea which popped into my head the previous day, which was: comparring all previous comparrisons on each iteration, on a basis of jumping between 2 graphs, in order to compare a route which goes through: A=>B=>C, and it's jumping equivalent to the route: A=>C, and so are the routes: B=>C=>D, and B=>D, thus forming a certain symmetry => I wasn't all too sure was going to lead me to the right solution, but was an idea worth exploring nonetheless.

I tried representing that symmetry as the 2-symmetric graphs idea we've previously seen in class, and it actually did give me some intuition cause I thought it would require building a special Dijkstra-like algorithm, which would run on a specially designed 2-symmetric-graphs configuration...

Spent an additional couple of minutes just planning the construction of such a graph-traversal-algorithm and came to the conclusion that I've complicated the entire thing a little too much:

the only thing a certain A[i]-element of the array requires in order to calculate the minimal route, is his two predecessors: A[i-1], A[i-2] compared by their own minimal routes, which both do represent the recursive structure I've shown previously (with odd/even configuration), and both elements do require the same recursive structure and comparison, essentially tying an entire route to any A[i] step of the array to be a combination of the minimal steps, taken at each "iteration" through an algorithm, from the base case- and building right up to A[i], with minimal route calculations present at each step of the array.

My fixed algorithm (using DP):

```
def targil_1_dyn(costs_list):
    # the two integers representing the min-route for each odd/even iteration of the loop:
    min_even: int = 0
    min_odd: int = 0

    # the two base-cases included in the loop:
    # entry at step=0:
    # enters the first if-block => min_even = min(0, 0) => min_even = 0 => min_even += cost (10 in our case) => min_even = cost (10).
    # entry at step=1:
    # enters the second if-block => min_odd = min(0, 10) => min_odd = 0 => min_odd += cost (11 in our case) => min_odd = cost (11).
    #
    # these account for the two stairs-entry states provided in the exercise.
```

```
    for step, cost in enumerate(costs_list):
        # print(f"step: {step}, cost: {cost}, min_even: {min_even}, min_odd: {min_odd}.")
        if step % 2 == 0: # even-ended-route:
            min_even = min(min_odd, min_even)
            min_even += cost
        else: # odd-ended-route:
            min_odd = min(min_odd, min_even)
            min_odd += cost

    return min(min_even, min_odd) # this part makes sure we return the smallest of the two sums, passing the n'th step.
```

Big-O-runtime calculation and optimization comparison:

the problem explicitly specifies the existance of a givennon-negative-array of size-n. all additional variables (min_odd, min_even) in my python-algorithm are of the same order of magnitude as O(1), and are allocated only once at the single call to the targil_1_dyn() function.

Total space complexity = O(1).

O(variable_allocation) = O(if-else_branching) = O(min()_function) =

O(even/odd_check) = O(min_even/min_odd value reassignment) = O(1).

The loop iterates-n times, once for each element of the array, and only consists of O(1) operations, regardless of the branching logic, which sums up to be a total of O(n).

Total time complexity = O(n).

When I finished building this algorithm I did pass it to gemini for an evaluation of correctness, and it did show me a more optimized version of this algorithm, which included using a prev2, prev1 variables instead of my odd/even idea, which did require constant branching-logic-comparison and the expensive modulo operation on each of the loop iterations, and instead: switching it to constant re-assignment of the prev2, prev1 variables, which completely blew my mind with it's similarity to the memoized- Fibonacci-sequence, and made total sense considering the insane initial Time complexity I ended up with, before I switched to reusing the previous min_routes of the array, which in a way did ended up memoizing all previous calculations of min_routes, thus going from fibonnaci's insane: O(2^n) exponential time complexity, all the way down to the linear: O(n).

Problem 2:

Important assumptions:

All values in the nums[] array are within the range: (1<=nums[i]<=101), which means they can't be negative.

The function of overall value is calculated by the summation of all bursts, which is the summation of all (nums[L] · nums[i] · nums[R]) calculations for any chosen element-i of the array, in a certain sequence combination of pops (dependant of the current nums[L], nums[R] elements of the array, in relation with-i).

Method: just giving up, this sounds too complex now that I understand that nums[L], nums[R] are moving independantly between iterations...

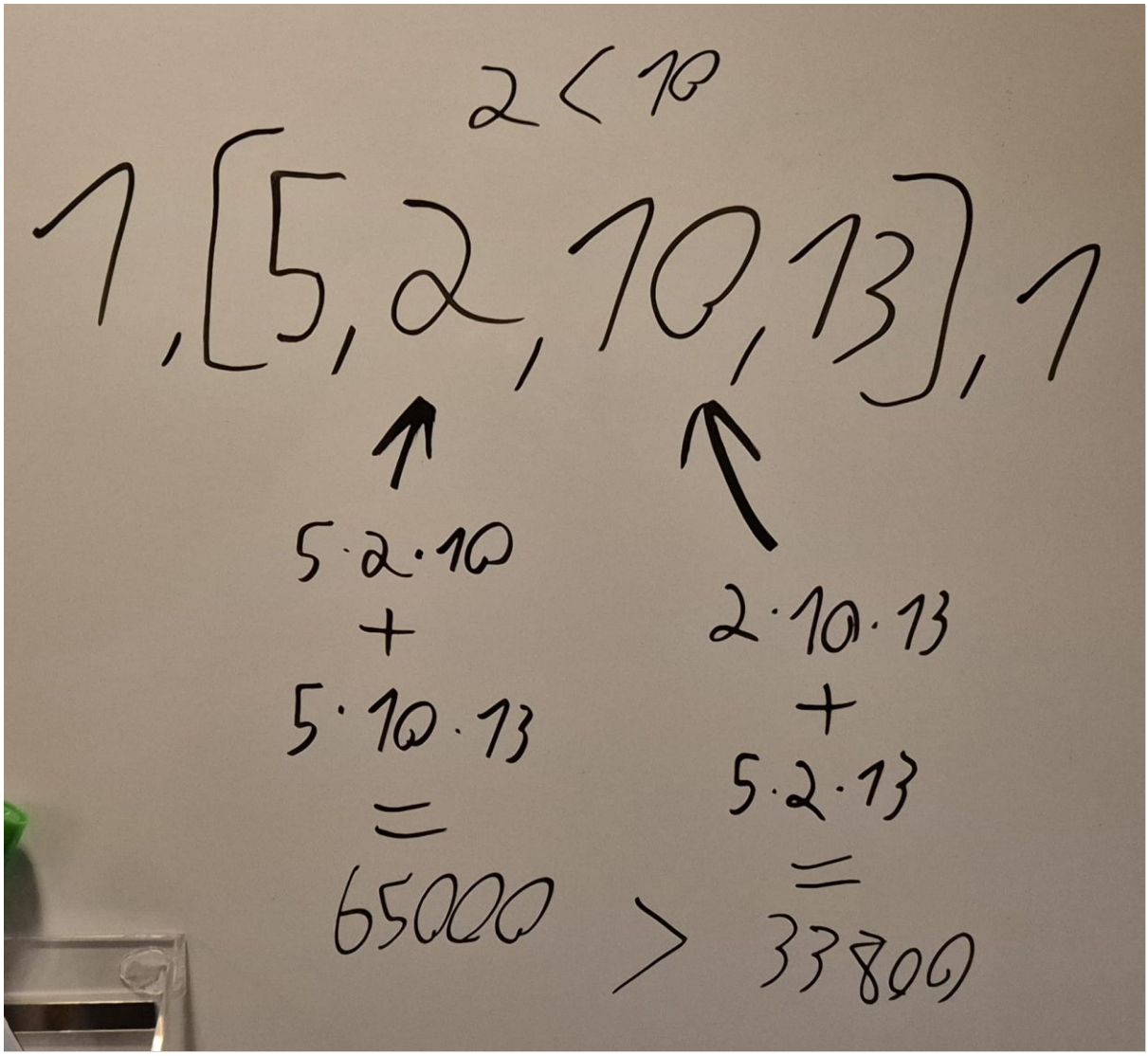
Actual solving attempts:

Preparation:

learning from Problem 1 I decided to spend more time thinking on the method of setting up the recurrence relationship between each iteration.

I did though have a rough time just visualizing the pattern without hands-on manipulation of the states and summed-up-values, which did require me to play with some numbers and try to build intuition, but yet keeping a healthy balance of time before I jump straight into building another poorly-planned algorithm.

Example-1:



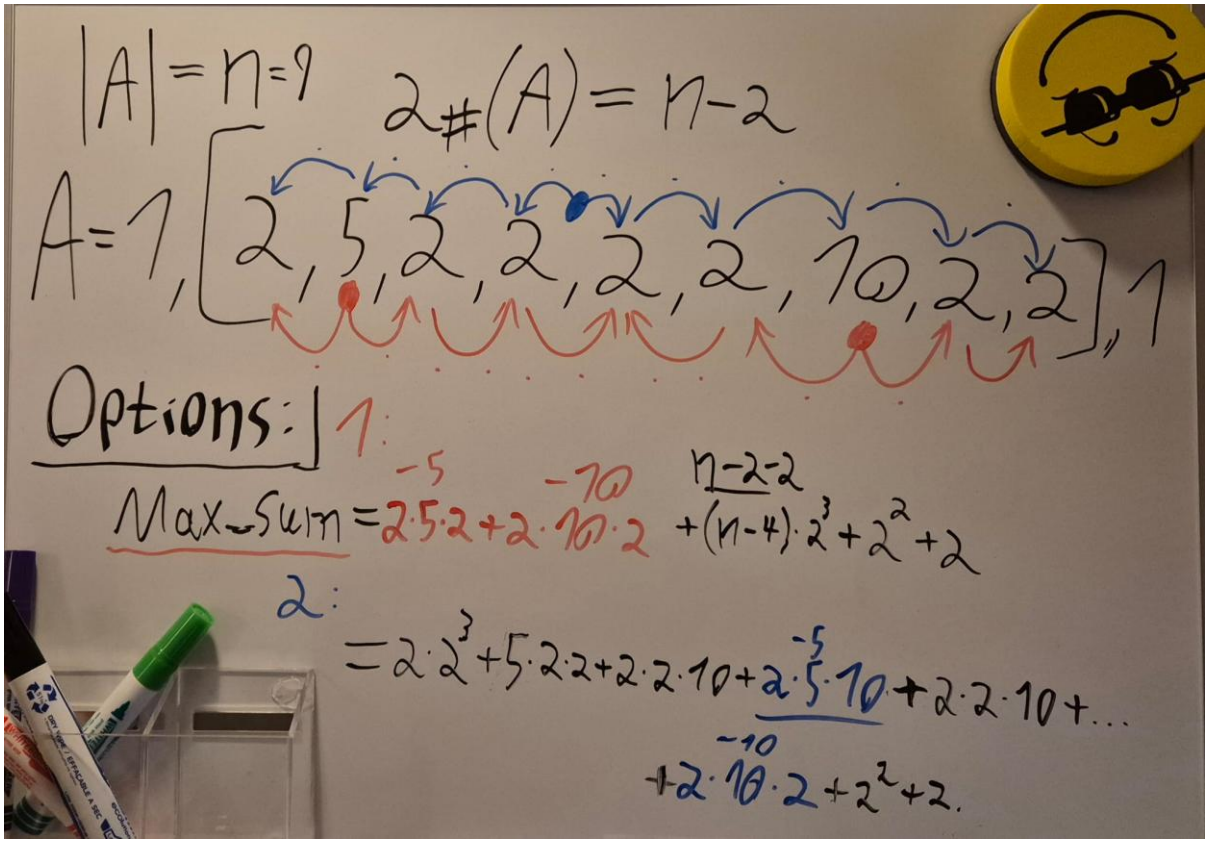
Disproved an early assumption I made about the best choice of element discarded being the largest element of the set.

The sequence of choices in this set shows that chosing the smaller middle element-2 (instead of the larger-10), significantly increased the overall sum, and found the best sequence of choices to get the maximum sum.

My initial assumption to this experiment was that by priorotizing getting rid of smaller numbers first, we keep the larger values alive for longer, thus allowing them (the larger values) to be multiplied more times, thus increasing the total and getting closer to the maximum.

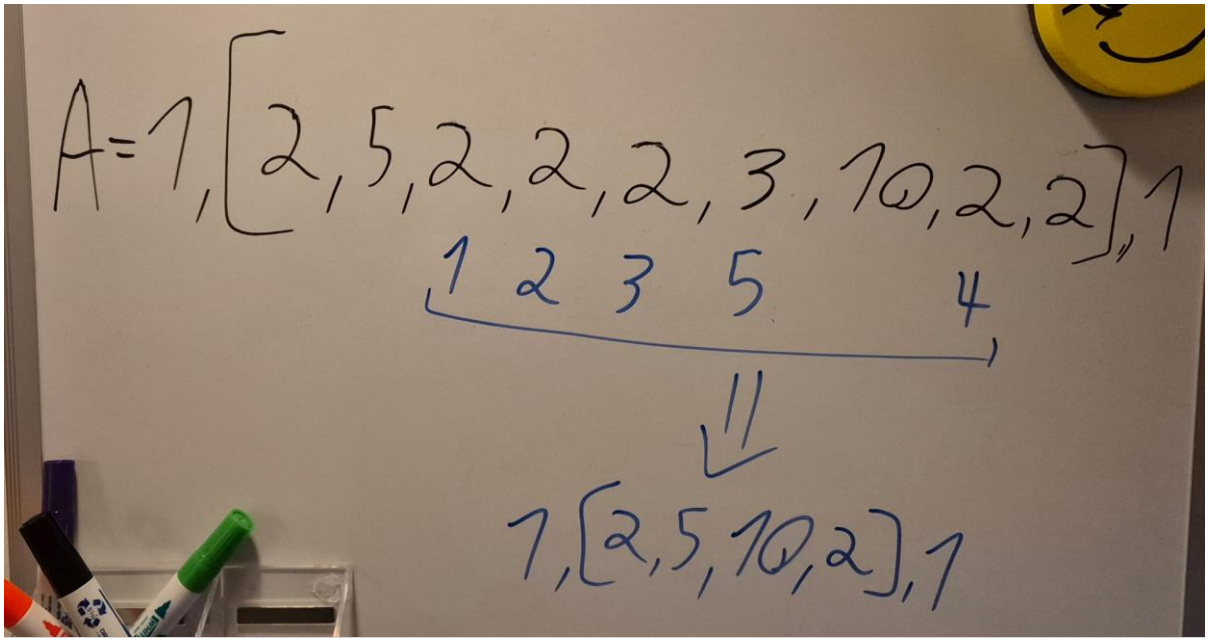
I was cautious of making early assumptions about a pattern that might aswell not exist, and might mislead me without proper proof of correctness.

Example-2:



This one further reinforced my intuition, and though not yet a full proof to the assumption, could be a viable pattern.

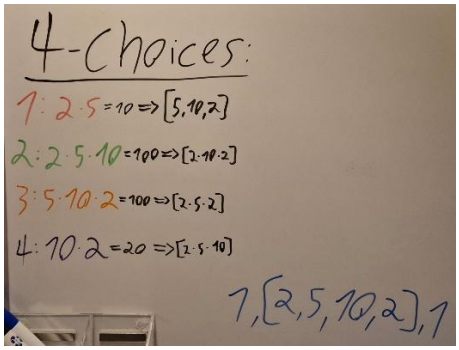
Example-3:



This one started all innocent: I was trying to expand on the previous example and try another intermediate value, larger than 2, that would sit in-between 5 and 10 and would further strengthen my intuition about the steps of recursion, being that we should always pick the smaller value of the set.

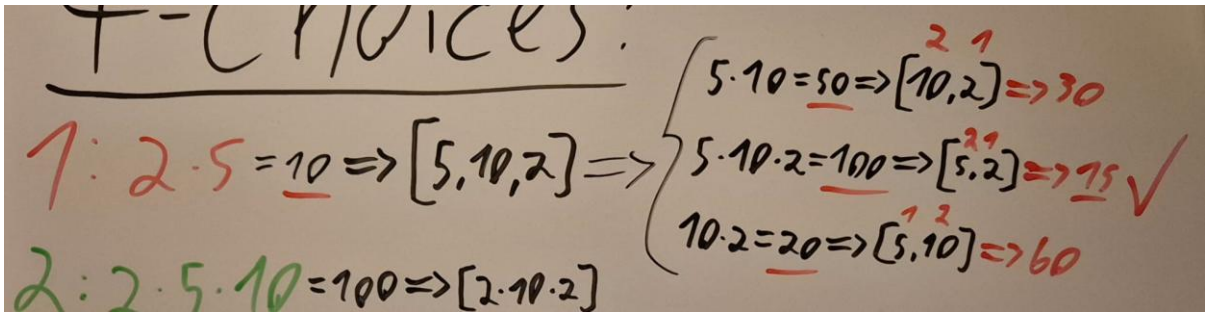
I tried a couple of sequences and started thinking that I might require another condition, that would make sure to maximize a local multiplication of balloons of the same values, as in the case here: we would prefer to pop a balloon that is close to 5 than to 3 or another 2.

So that got be choosing the above sequence, and one thing let to another decided to go overkill and calculate the best choice of the tiny set: [2,5,10,2], expecting it to reinforce further initial belief, and maybe help me get closer to a formal proof by induction where I can start at a simple base case, similar to this, and start expanding:

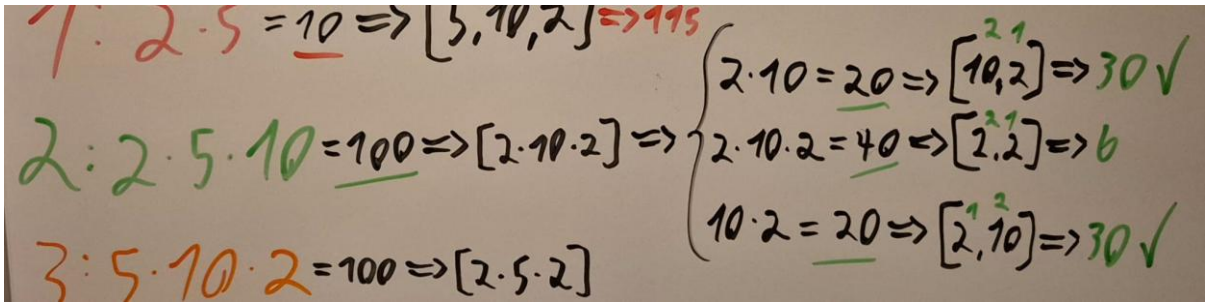


Now what I didn't expect is just how much of a difference every little choice, on every iteration would make when it comes to the best balloons to choose, what would debunk my simple initial assumptions of #1: “longevity of larger values”, and #2: “proximity to larger values”, being the only conditions that would find that one max_total_sum:

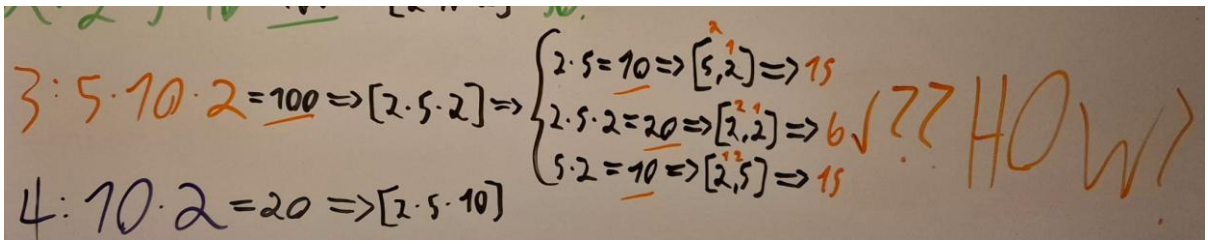
1.



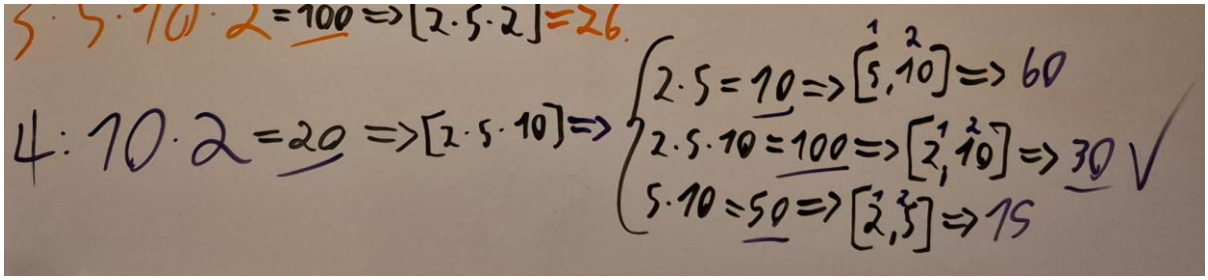
2.



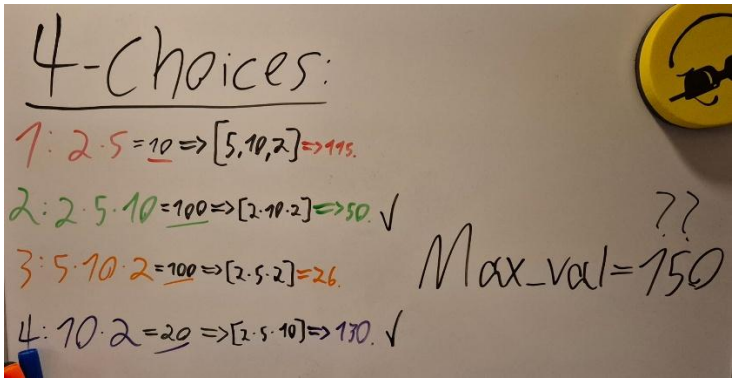
3.



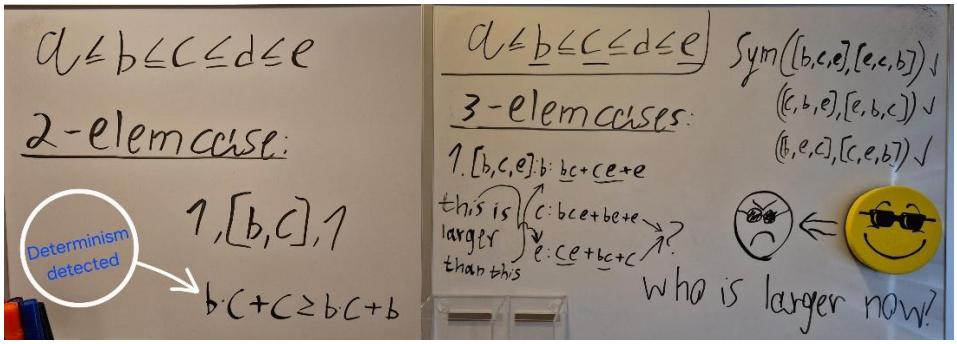
4.



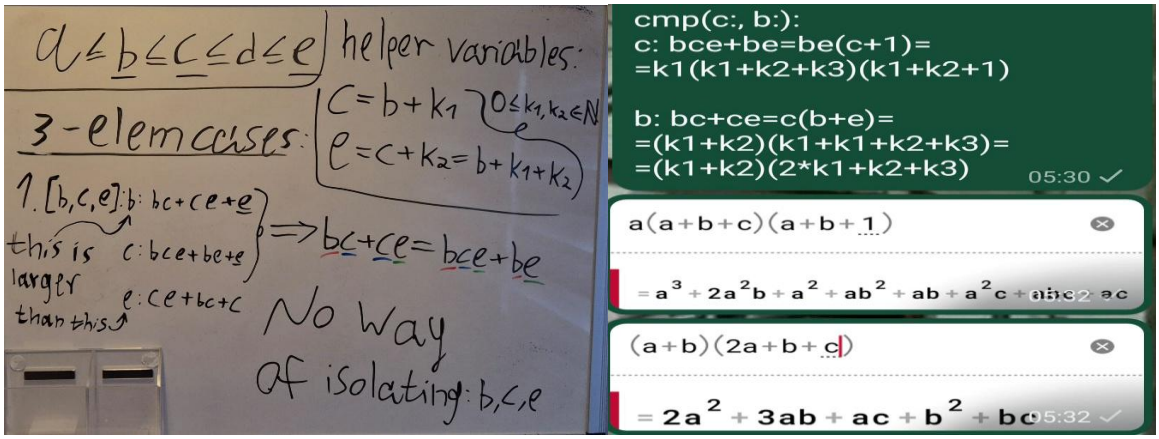
Max:



Now apart of just flexing my new board and color pens, a physical trial and error has shown me that up until the point when the set held only 2 balloons, I had no deterministic way of making the best choice at each iteration. A generalization of the smaller array cases has further strengthened my suspicion:



In the second picture, the “b:” statement is actually larger than the “e:”.



Found no way of isolating any of the variables to find a simple algebraic pattern of sizes. Declaring k1,k2 for discrete calculations of the distances between 2 variables didn’t seem to help either… maybe the use of discrete generating functions could be useful here(?) maybe not(?)…

This meant my initial approach of finding a general-case pattern for the best sequence of balloons popped did in-fact mislead me into thinking a couple of simple rules could apply to this problem which is exponentially growing in logical complexity and requires handling every single iteration with a completely different approach, since they weren't independent of each other.

Apart of just wasting my time, crying over getting stuck in deep mathematical logic (which is probably way above my level of knowledge) wouldn't be productive, so I decided on getting my hands dirty by building a naive (recursive) version of the algorithm that would compare all subcases of the problem from top to bottom and return such a max_total value and the sequence that generated it:

```
def targil_2_naive_wrapper(nums, shuffle: bool):
    if shuffle:
        random.shuffle(nums)
        print(f"shuffled nums: {nums}")
    return targil_2_naive_rec(nums)

def targil_2_naive_rec(nums):
    nums_length = len(nums)
    # print(f"nums: {nums}, balloons_left: {nums_length}")
    if nums_length == 0: return 0, []
    if nums_length == 1: return nums[0], [nums[0]]

    largest_score = 0
    best_sequence = []

    for index, value in enumerate(nums):
```

```
    if index == 0: # current_calc = 1 * nums[0] * nums[1]
        middle_mul = value * nums[1]

    elif index == nums_length - 1: # current_calc = nums[n] * nums[n-1] * 1    (n = balloons_left)
        middle_mul = value * nums[nums_length - 2]

    else: # (0 < index < balloons_left):
        middle_mul = nums[index - 1] * value * nums[index + 1]

    left_nums = nums[:index]
    right_nums = nums[index + 1:]

    # print(f"index: {index}, nums: {nums}, balloons_left: {nums_length}, left_nums: {left_nums}, right_nums: {right_nums}")
    left_score, left_seq = targil_2_naive_rec(left_nums)
    right_score, right_seq = targil_2_naive_rec(right_nums)

    current_score = left_score + middle_mul + right_score

    if largest_score < current_score:
        largest_score = current_score
        best_sequence = [value] + left_seq + right_seq # concatenation of lists in  python.
  return largest_score, best_sequence
```

The algorithm even utilized randomization of order in the original nums[] array, and playing with the size of generated values didn't help find a pattern I could categorize and assume inside recursive conditioning.

Actually, it had the reverse effect: the more naive of an assumption I would have about the order of sequence, the easier it would be contradicted by a

different random generation of order/size/magnitude of values, or their difference in distance... all effecting the relation between multiplication and summation in a matter that was unpredictable. Hell, I even considered a discrete-convolution method, seeing all the complex multiplication-summation operations happening led me into thinking it would require much more time that I had in order to potentially some day come across a certain pattern into increasing the accuracy of my manual predictions of an optimal max-sequence.

It was about that time I started giving up on the recursive method by using a certain mathematical ordering, assuming it was even possible proving that an optimal sequence of such sort exists and can be evaluated when presented with the array's values, longer before actual calculation attempts take place... the more I thought about the amount of complexity I'm facing, the further I realized I'm nowhere close to the required O(m^3) time complexity I was asked to match, and any further mathematical experimentation would only draw me further from finding the correct algorithm.
That was when I had given the idea of “dynamical programming” a fair amount of thought and started thinking of alternative ways that could reduce the exponential time of the recursive method by “starting from the ground up” or whatever that meant.

I had an idea of approaching DP method by calculating the largest scores at each iteration, from bottom-up, which meant starting with the smallest subsets of numbers possible and comparing them on each level.

A nice fallback thought I had was that in order to match the possible comparison of any 2 numbers in their multiplication with 1 in an array of size-1 (which happens to be a possible state to end up in on any given set of numbers), I would construct a fully-connected graph, containing all original numbers of the array (m nodes), and compute the relations of multiplication between each pair of numbers in that graph (O(m^2) time). That way I would deterministically make sure that all lowest levels of the recursive algorithm are pre-calculated before the algorithm can proceed

onto the next layer (arrays of 3 numbers), and use the calculations from the previous layers any time a lower-sized computation is required- that way we save on re-computing the lower sizes of arrays, and might cut a significant portion of time complexity.

That being said, the new algorithm didn't actually require a graph being created, or any other ideas I had like using a modified “Floyd-Warshall”-algorithm, in order to find the largest computed “paths”, since the problem did require accounting for a specific order of execution.

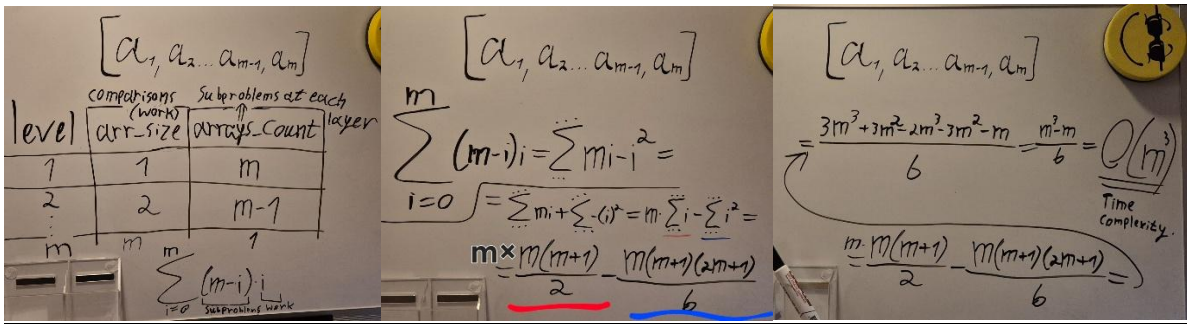
That meant that if the array contained: [1,2,3] as it's values, at no point could we change the order of computation to have 3 be present in the middle of 1 and 2, and even though a further more complex symmetry property would exist, that would (again) only add up to additional complexity I wasn't going to find this time of facing this problem (BUT MAYBE NEXT TIME...).

Anyways, the newly improved DP algorithm looks like this, and required using a built-in python cache in order to keep smaller array computations at each step of the iterations through larger array sizes:

```
def targil_2_DP_wrapper(nums, shuffle=False):
    if shuffle:
        random.shuffle(nums)
        print(f"shuffled nums: {nums}")

    padded_nums = [1] + nums + [1] # gemini recommended doing the 1' paddings instead of introducing =>
    # unnecessary branching logic inside each iteration itself, which slightly increases overhead.

    @lru_cache(maxsize=256) # limiting the maximum cached number of calls. =>
    # for the assignment's time and space complexity, we assume that the cache is of infinite size.
    def dp(left, right):
        max_coins = 0
        best_sequence = []
```

```
    for i in range(left + 1, right): # this range makes sure to account the left-padding on each iteration.
        left_score, left_seq = dp(left, i)
        right_score, right_seq = dp(i, right)
        current_coins = left_score + (padded_nums[left] * padded_nums[i] * padded_nums[right]) + right_score

        if  max_coins < current_coins:
            max_coins = current_coins
            best_sequence = left_seq + right_seq + [i]

    return  max_coins, best_sequence

  result = dp(0, len(padded_nums) - 1) # =>
  # executing the first dp() call on the full nums[] array.

  #-----{
  # doing this to tie dp()'s lru_cache's methods to the wrapper function's for evaluation in main():
  targil_2_DP_wrapper.cache_info = dp.cache_info # pyright: ignore[reportFunctionMemberAccess]
  targil_2_DP_wrapper.cache_clear = dp.cache_clear # pyright: ignore[reportFunctionMemberAccess] =>
  # the pyright linter really doesn't like attaching new function attributes... (static-typing FTW!!)
  #-----}

  return result
```

Big-O-runtime calculation and optimization comparison:



Assuming the problem provides the array as it is, the only thing required in terms of memory allocation is comparing all possible sizes of any subsets: 1<=possible_sizes<=m, for each of the-m recursive iterations throughout the dp() function. This could be calculated by the sum in the images I took, or better yet treated as shifting possible boundaries between the 2 walls(variables): left,right, and turned into a combinatorics calculation represented by: C(m,2), where 2 is the 2 boundaries and m is the total size of our set (total size of the array).

Total time complexity = O(m^3).

Total space complexity = O(m^2):

Could be represented by a dictionary cache, encoding the left,right boundaries as the key entries, and the results of each calculation as the value,

or for an even cooler method: using an upper triangular matrix, tracking the left,right coordinates as rows(i), columns(j).

either way it is bounded by O(m^2).

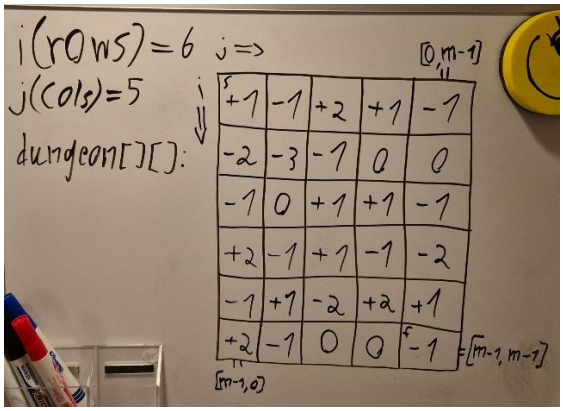
Problem 3:

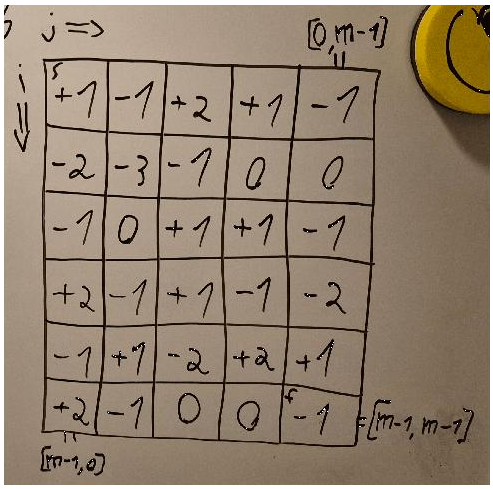
Decided on speeding up with this problem, since I already knew this one.

My instinct was to to use the Bellman-Ford algorithm in order to traverse the grid as a graph containing no-circles, but some potential negative values, which removes the ability to use Dijkstra in this case.

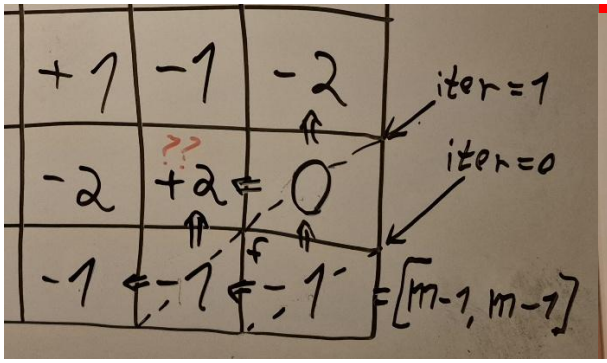
Bellman-Ford is a dynamic programming algorithm, so apart of replicating the algorithm into python, the only thing that was left to do was to ensure we build the correct edges, that would match the problem's pathing requirements, and simply run the algorithm.

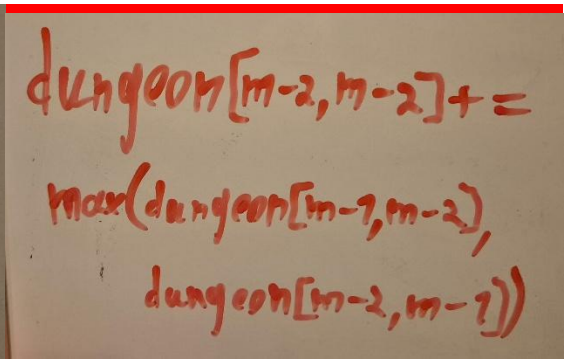
The entire next section is wrong and is based on faulty understanding I had at the time of writing the first algorithm... I explain later on at-(#1):





The comparison logic at iter=1:





The total amount of iteration across the *anti-diagonal* is equal to rows+cols-1.

I got entangled in flawed structural logic in my first trial of building this algorithm (as I explain later on),and came to the conclusion that trying to "fix" this terrible mess using AI would be pointless as it would require structural changes that would overcome my will to compare this version to the better one I will later construct.

I left it as it is in the original python file, simply for additional documentation of mistakes made in the process, and the learning curve I had to overcome when trying to conceptually grasp this idea of DP.

(#1) Additionally, I would only later learn that my initial understanding of the requirements was wrong, as I assumed I needed to calculate the largest path as a whole (the max total health), instead of the local calculations for tracking the health at each step for being above-0...

Decided on making a better one which actually utilized the m, n sizes instead of padding a matrix and getting entangled in conflicting motives of traversing a matrix which can be padded in different ways and be of different sides of 'None' lines, adding additional conditions to check for no out-of-bounds access... while still utilizing "self-updating" of the values of each entry in the matrix instead of allocating an array the size of the largest "anti-diagonal" in the matrix.

In addition, my mixed-approach between using both recursion and iteration by indexes turned out to be an entire mess of conflicting objectives which stemmed from my lack of understanding of the optimal dynamic-programming approach to this problem at the time.

While my first attempt of traversing using loop-iteration could be done using another loop, much more efficient conditional-logic and 0 recursion, and it would still be DP, I decided it was worth following the requested creation of a recursive dp() function, as the problem suggested, even though that would (**again**) take my longer than I anticipated and would make me regret not using iterators, as the traversal I had to use for the recursion is pretty much a well-ordered sequence of updating values, since no parallel-running approach I thought of could satisfy the requirement of the already pre-calculated deterministic results in a bottom-up approach...

anyways, here's the code:

```
def targil_3_DP(dungeon, rows, cols, shuffle=False):
    if  shuffle:
        np_matrix = np.array(dungeon)
        flattened = np_matrix.ravel()
        np.random.shuffle(flattened)
        rows, cols = np_matrix.shape
        dungeon = flattened.reshape(np_matrix.shape).tolist()
        print(f"shuffled dungeon[][]:")
        for row in dungeon:                                    # pyright: ignore[reportGeneralTypeIssues]
            print(row)

    dp3(dungeon, rows, cols, rows - 1, cols - 1) # =>
    # the first dp3() call will start at dungeon[rows-1][cols-1]. on each call of =>
    # dp3(), we check the-2 (possible) blocks: below, and to the right, of the =>
    # current location in the matrix, append the min() of the 2 to the current =>
    # block, and keep moving and comparing until we get to dungeon[0][0].

    return dungeon[0][0]                                       # pyright: ignore[reportIndexIssue]


def dp3(dungeon, rows, cols, current_row, current_col):

    # a single updating of the dungeon[rows-1][cols-1] block:
    if  current_row == rows - 1 and current_col == cols - 1:
        dungeon[current_row][current_col] = max(1, 1 - dungeon[current_row][current_col])

    else:
        # assigning the below/right blocks and checking boundaries:
        below_block = dungeon[current_row + 1][current_col] if current_row + 1 < rows else math.inf
```

```
        right_block = dungeon[current_row][current_col + 1] if current_col + 1 < cols else math.inf


        # calculating minimum required health from the current-block to dungeon[rows-1][cols-1]:
        dungeon[current_row][current_col] = max(1, min(right_block, below_block) - dungeon[current_row][current_col]) # =>
        # took me hours until I realized I was calculating the maximum path length and not checking health =>
        # bounds on each iteration like a real videogame smh...


    # base case checking condition:
    if  current_row == 0 and current_col == 0:
        return # we've arrived at dungeon[0][0] and should return to targil_3_DP().

    # new dp3() calls:
    #-----
    go_up = (current_row!=0) # all blocks call up.
    go_left = (current_col!=0) and (current_row == rows - 1) # making only current blocks which are in =>
    # row: dungeon[row-1] make 2 calls. any other block will only call on the block above him, making an =>
    # "activation chain" looking-pattern where only the lowest block activates an entire column, making sure =>
    # every comparison of previous below/right blocks has already been updated, and never calling twice on the same block.


    if  go_up:   dp3(dungeon, rows, cols, current_row - 1, current_col)
    if  go_left: dp3(dungeon, rows, cols, current_row, current_col - 1)
    #-----
```

Correctness and DP-intuition:

Pretty sure I went over everything in the gemini conversation, and in the comments in the code but to recap:

the DP approach kind of blew my mind when I finally got it in that second code of Problem_3, and finally made sense how even though the formula of the health required isn't symmetrical, we assume the location at a certain block has been reached, and traverse the same direction into dungeon[rows-1][cols-1], making it a 1.deterministic, 2.inductive, 3. Non-repeating(this one is important for the time complexity compared to the top-down recursive method) calculation, at every new step of the way, each time expanding the deterministic and highly efficient calculation until finally hitting that start position.

Funny enough I thought this one will take me the least amount of time but took me just as long as Problem_2, and required a serious amount of concentration and innovative thinking, even though I swear I was solving these sort of problems in the past just based on intuition, long before I knew the names and methods... not an easy assignment, not at all.

Big-O-runtime calculation and optimization comparison:

The optimization part in this problem is mainly focused on the size of additional memory required.

As I explained before, I decided to not allocate any linear memory apart of local variables O(1), and that's because I have overridden every current-block in the original matrix, since that original value per block was only useful to calculate the minimum required health at each step of the backtracking sequence in the DP recursive process.

The suggested way of solving this while using O(n) space would probably mean using an array the size of columns in the matrix, and updating each entry to that array in the corresponding current-block's column-entry.

That approach seems much less complicated than I imagined it when I was (wrongly) trying to iterate over the anti-diagonal of dungeon[][], but what the hell, I managed to save an-O(n) amount of space..

Total space complexity = O(1).

The time complexity in this algorithm is more straight forward, each block in the matrix is review once and compared using O(1) operation to the 2 previous blocks (below_block/right_block), for m*n overall iterations through the algorithm as a whole, for the size of dungeon[][].

m*n*O(1)=O(m*n).

Total time complexity = O(m*n).

Problem 4:

Got stuck on the semantic part with this one and problem 5. Would appreciate problem 4 explicitly stating that prices[] holds the record for future prices... would have make it a little easier to understand, plus giving away the methodology with the 3 constant variables sort of threw me off.

+ “You may buy and sell the stock multiple times” can really have different meanings depanding on context but doesn’t matter now, Here's the code:

```
def targil_4_DP(prices, Rest, Sell, Buy, shuffle=False):

    if shuffle:
        random.shuffle(prices)
        print(f"shuffled prices: {prices}")

    Rest_prev = Rest
    Sell_prev = Sell
    buy_prev = Buy

    for price in prices:
        Rest = max(Rest_prev, Sell_prev)
        Sell = buy_prev + price
        Buy = max(buy_prev, Rest_prev - price)

        # updating prevs:
        Rest_prev = Rest
        Sell_prev = Sell
        buy_prev = Buy

    return max(Rest, Sell)
```

Big-O:

Comparing and updating the current constants based on the prev-constants and other conditions (like the cooldown) take O(1) time, we're doing this a total of n time for traversing the entire array: n*O(1)=O(n).

Total time complexity = O(n).

Only constants are assigned, constants take O(1) space.

Total space complexity = O(1).

Problem 5:

Problem 5 was extra hard, took a really long time. I even spent some extra amount of time generating a gantt-chart to visualize the idea since it was just too hard to grasp.

Here's the code:

```
def targil_5_DP(times, jobs, profits):

    # restructuring the data into lists: entries = [(start, end, profit, original_id), ...]
    entries = [(times[i][0], times[i][1], profits[i], i + 1) for i in range(jobs)]

    # sorting jobs by end-time (Ascending):
    entries.sort(key=lambda x: x[1])

    sorted_end_times = [entry[1] for entry in entries] # this is going to be =>
    # the list used to binary search jobs and then fetching the coresponding job entry.

    # cache that store the currently maximum profit achievable:
    dp_cache = [0] * (jobs + 1) # includes the base: 0 jobs (= 0 profit).


    for i in range(1, jobs + 1):

        current_start, _, current_profit, _ = entries[i - 1]

        # using binary-search to find a job that ended BEFORE OR AT our current_start.
        # the built-in bisec_right returns an index indicating the jobs to the left of =>
        # that index in- sorted_end_times are all compatible with the condition.
```

```
        compatible_job_idx = bisect.bisect_right(sorted_end_times, current_start)

        #splitting options comparison (2 per each entry in: entries[i] ):
        #---
        # option A: Exclude current job (take previous max):
        exclude_profit = dp_cache[i - 1]
        # option B: Include current job (take its profit + max profit of previous (time compatible) max profit):
        include_profit = current_profit + dp_cache[compatible_job_idx]
        #---

        dp_cache[i] = max(exclude_profit, include_profit)

    backtracking = []
    current_i = jobs

    # we traverse the DP array backward, starting from the final calculated maximum.
    # at each step, we determine if the current job contributed to the total profit:
    while current_i > 0:
        current_start, _, _, original_id = entries[current_i - 1]

        # if the profit is greater than it's previous we know that: =>
        # that job's profit must have been included:
        if  dp_cache[current_i - 1] < dp_cache[current_i]:
            backtracking.append(f"J{original_id:02d}") # => this job was chosen.
            # jumping to the latest compatible (non-overlapping) end_time:
            current_i = bisect.bisect_right(sorted_end_times, current_start)

        # and if the profit is equal to it's previous then: =>
```

```
        # that job's profit was skipped by the algorithm (a better sequence of choices was chosen instead of it.)..
        else:
            current_i -= 1 # incrementing the index to check the next profit's comparison and so on...

    return dp_cache[jobs], backtracking
```

Big-O:

Worst sorting time: O(n*log n).

In the worst case senario of running the binary search algorithm we will have to run for O(log n) time, for n iterations: O(n*log n).

All additional comparison logic takes O(1), for n iterations that's: O(n).

O(n*log n) + O(n*log n) + O(n) = O(n*log n):

Total time complexity = O(n*log n).

Only linear space complexity ADTs were assigned, like the cache:

Total space complexity = O(1).

Extension:

I had the idea of using more constant conditions and linear cache space to make additional comparisons in order to reduce even more dimensions of complexity like 'difficulty' or 'budget', and maybe find a smart way of reducing dimensionality like using a PCA algorithm or something else but no algorithm that I know of can reduce dimensionality without adding noise to the system (debated about this idea with gemini, he explained the idea behind FPTAS, which somewhat relates to my idea, but never solves the problem in a 100% deterministic manner).

The pattern I noticed when doing this assignment is how the more complex DP problems tend to have independent constraints or conditions, that can influence the future outcome of a certain path in non-deterministic ways, forcing the naive (recursive and not-cached) approach of checking every single combination or route, which would explode in complexity and be extremely inefficient.

That pattern extends with the new “extra“ independent conditions and ties back into the way we so that being implemented in class: how a discrete boundary or a threshold of 'B' choices forces us to check every single inner combination of the 'n'-original states, with the 'B' added states, turning this into a matrix of possibilities, all independently influencing the goal of maximizing the total profit.

In our current problem, 'n' represents the total number of jobs we have to check, 'B' can represent the overall budget, 'C' might represent some sort of difficulty metric... forcing the total calculation into the lower limit of O(n*B*C), even though the original problem WAS able to find a pattern on the time-metric that forces conditions on jobs, but the time metric wasn't an independent factor in the problem- that’s an important point I had to wrap my head around, to finally understand the real scaling factors of problems of this sort in dynamic programming, and their real lower boundaries, compared to the implicit patterns that allow it to reduce some complexity, and still not violate the independent choices that linearly scale the complexity.

Gemini conversations links:

Disclosure: I didn't fully read every single output, but rather used it briefly in periods of needing confirmation of steps, strengthening intuition, asking for directions when I got stuck, or generating some code when I was getting too tired for the day.

Also, all links were generated at the end of the conversations, and no additional prompts were created, that's the entire record for my use of AI in this assignment, every single prompt, brainstorm, or shortcut I used.

Problem_1: https://gemini.google.com/share/88b12bee6d82

Problem_2: https://gemini.google.com/share/416b10fbaaf1

Problem_3: https://gemini.google.com/share/8c4904ca3ccc

Problem_4: https://gemini.google.com/share/c7b0b1829423

Problem_5: https://gemini.google.com/share/e0a9f40dbc84